

VCKss, Mata, and MATLAB at Scale

A source-bound time and memory guide for applied researchers

VCKss development benchmark

29 August 2026

Abstract

This report compares the public VCKss Mata route, the qualified VCKss Rust `CMG_FULL_V2` route, and maintained MATLAB `LeaveOutTwoWay` over four graph families, five stored-row counts, and 1, 2, 4, 8, and 16 target-core cells. Rust and MATLAB use the full target; Stata and Mata use 1, 2, 4, 4, and 4 effective processors. Each cell has three position-balanced seed repetitions. Within every SCC task, all three routes run sequentially on the same scheduler-assigned host, so paired route ratios remain comparable without waiting for a particular node. Primary outcomes are complete estimator command time and peak summed process-tree resident memory during the estimator phase. The accepted evidence contains 297 complete same-host tasks and 891 successful fresh-process estimator calls. Three additional MATLAB attempts are right-censored at 10,800 seconds and retained separately.

Which route should I use?

There are 99 complete, rankable graph-size-core cells. VCKss-Rust is fastest in 99, VCKss-Mata in 0, and MATLAB in 0. These counts are a map of the measured grid, not a general language ranking. Use the cell-specific curves and guidance files:

- Use **VCKss-Rust** for the qualified match-deletion JLA tuple when its plugin is available and its receipted memory fits the budget.
- Use **VCKss-Mata** when source-only portability, inspection, or unsupported Rust options dominate speed.
- Treat **MATLAB** as a maintained cross-package comparator with a distinct RNG and numerical policy, not as an identity oracle.
- Prefer the smallest effective core count within 10% of the route's measured maximum when additional cores buy little speed. That maximum is four for Mata and 16 for Rust and MATLAB.

1 Registered workload and scientific contract

The graph families are strong degree 2, strong degree 3, strong degree 6, and a weak degree-3 shallow hub-tree leaf-panel bottleneck design. Stored rows are 7,680; 30,720; 122,880; 491,520; and 1,966,080. Fixed row counts imply different numbers of workers and firms across degrees and topologies. Every route receives the same literal CSV, 200 JLA probes, match deletion, joint nuisance handling, movers, observation-key ordering, no controls, and no weights.

Table 1: Registered benchmark dimensions.

Dimension	Registered values
Graph family	strong d2, strong d3, strong d6, shallow-tree/bottleneck d3
Stored rows	7,680; 30,720; 122,880; 491,520; 1,966,080
Target-core cells	1, 2, 4, 8, 16
Effective cores	Rust/MATLAB: 1, 2, 4, 8, 16; Stata/Mata: 1, 2, 4, 4, 4
Repetitions	3 seeds with position-balanced route order
Accepted calls	297 same-host tasks $\times$ 3 routes = 891
Censored calls	3 MATLAB attempts with time $\geq 10,800$ seconds
Projections	JLA P200

VCkss omits `tolerance()`, retaining production defaults of 10^{-10} for fit solves and 10^{-6} for randomized probe solves. The strict Rust call must return `CMG_FULL_V2`; the Mata call uses explicit `backend(mata)` `rng(stata)`. Successful VCkss calls must pass the complete original-system residual, accounting, target identity, `e(sample)`, data/RNG/sort restoration, memory, route, and lifecycle gates. Rust–Mata corrected targets use the independent-probe envelope

$$|\hat{\theta}_R - \hat{\theta}_M| \leq \max \left\{ 10^{-8} \max(1, |\hat{\theta}_R|, |\hat{\theta}_M|), 6\sqrt{\text{MCSE}_R^2 + \text{MCSE}_M^2} \right\}.$$

MATLAB must return finite targets, its accounting identity, the registered retained rows, and an accepted numerical status, but target gaps are descriptive. In the largest strong-degree-two one-core cell, MATLAB reached the registered 10,800-second limit in all three position-balanced repetitions. Those calls are right-censored lower bounds. The entire graph–size–core cell is excluded from paired rankings and speedup calculations; partial timings from the sequential tasks are not substituted for complete same-host triplets. The high-target Rust–Mata cells compare full-target Rust with capped four-core Mata. They are paired availability comparisons, not equal-core scaling claims.

2 End-to-end time

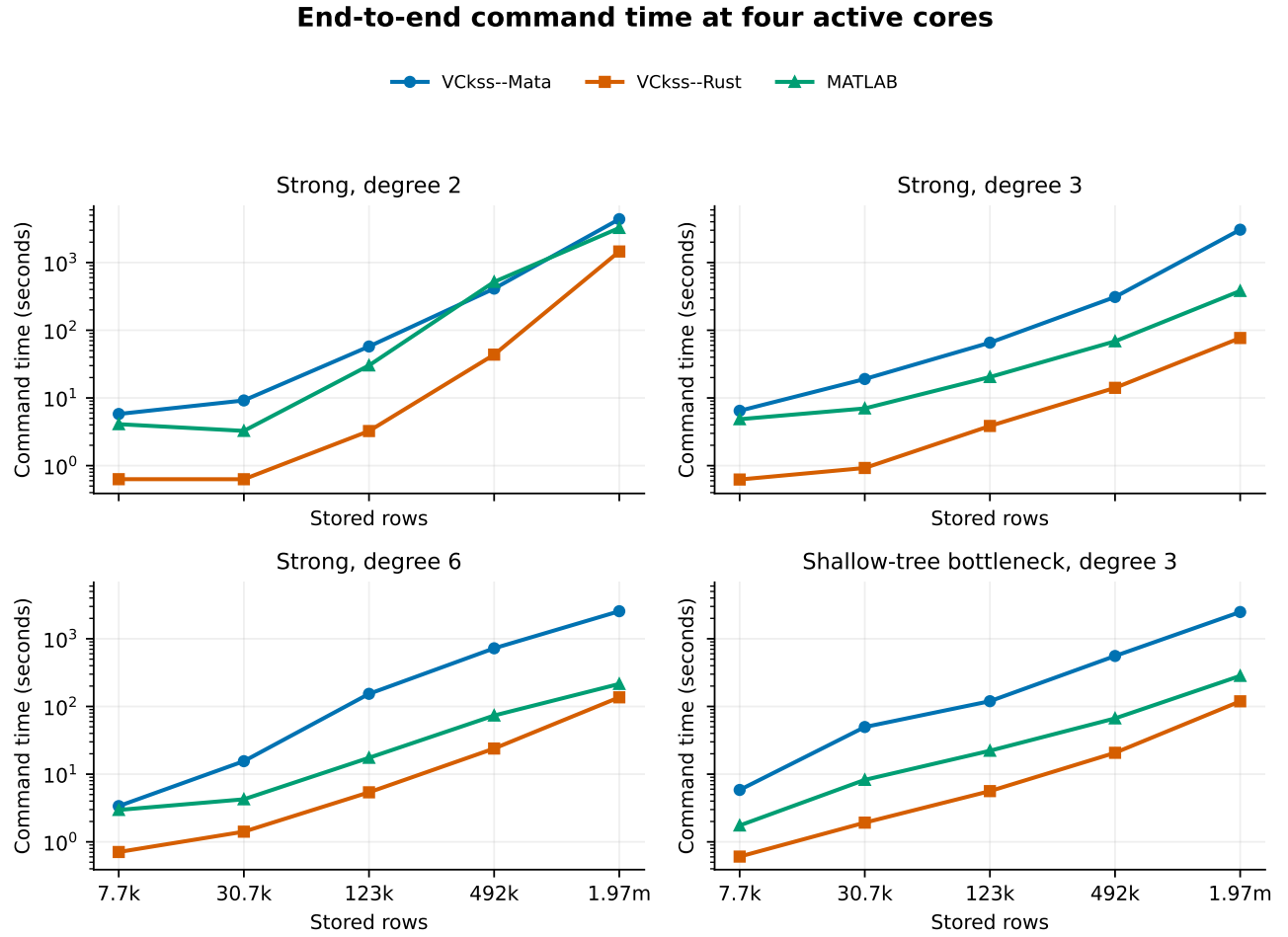


Figure 1: Complete estimator invocation-to-return time at four active cores. Lines appear only for complete, rankable cells. Both axes are logarithmic.

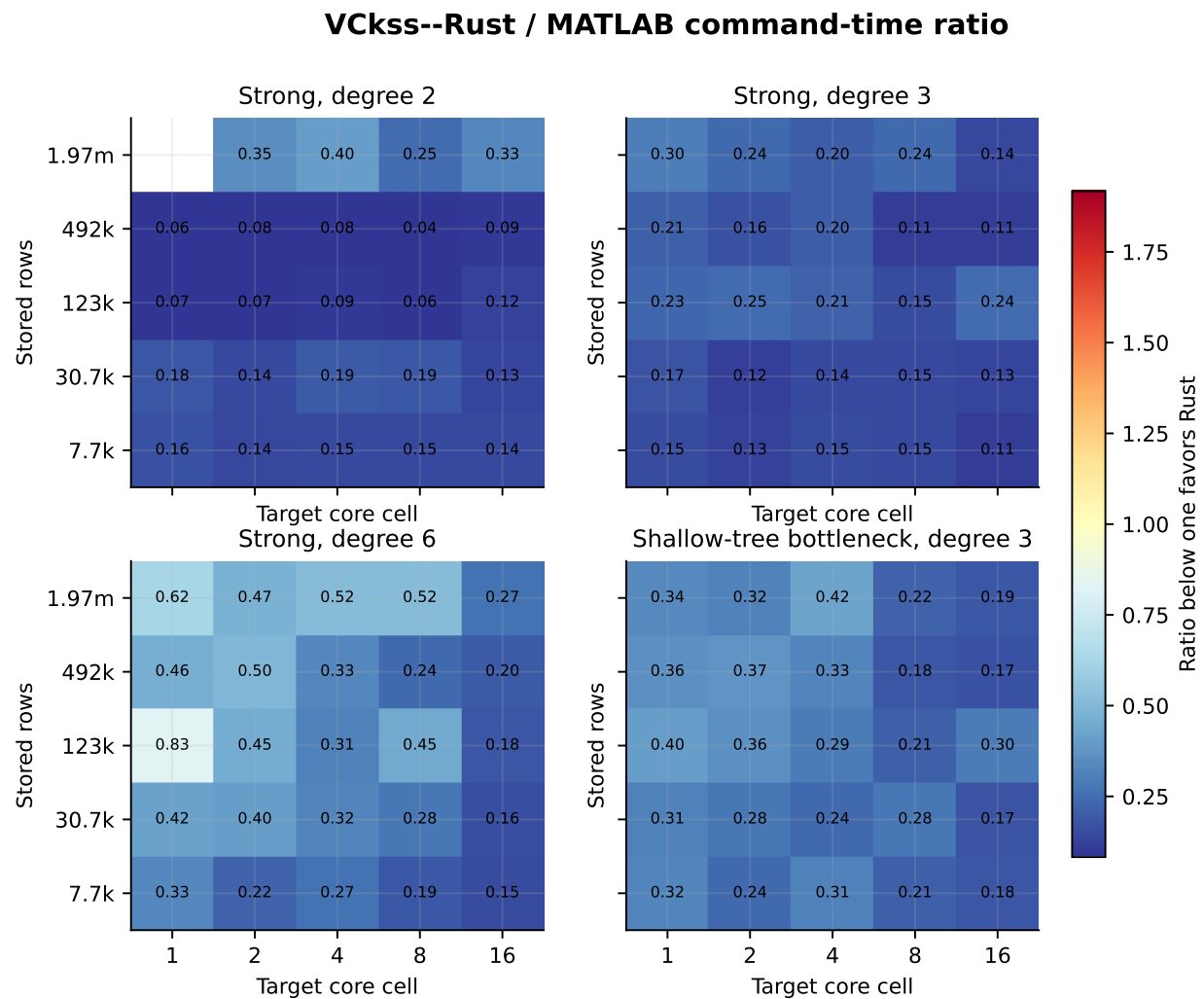


Figure 2: Median Vckss--Rust / MATLAB command-time ratios. Values below one favor Rust. Unrankable cells are blank.

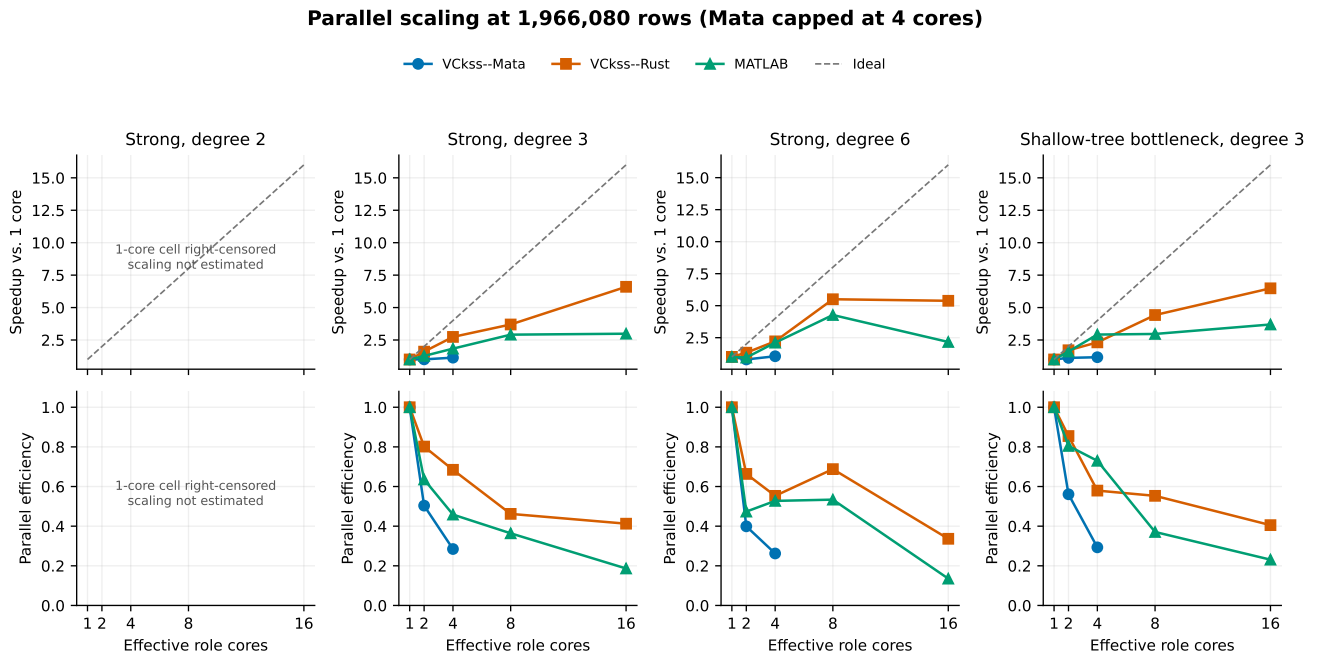


Figure 3: Speedup and efficiency over each route’s effective-core range at the largest measured row count. Mata stops at four cores; Rust and MATLAB extend to 16. The dashed line is ideal linear speedup. Curves that span CPU models are descriptive rather than controlled hardware comparisons.

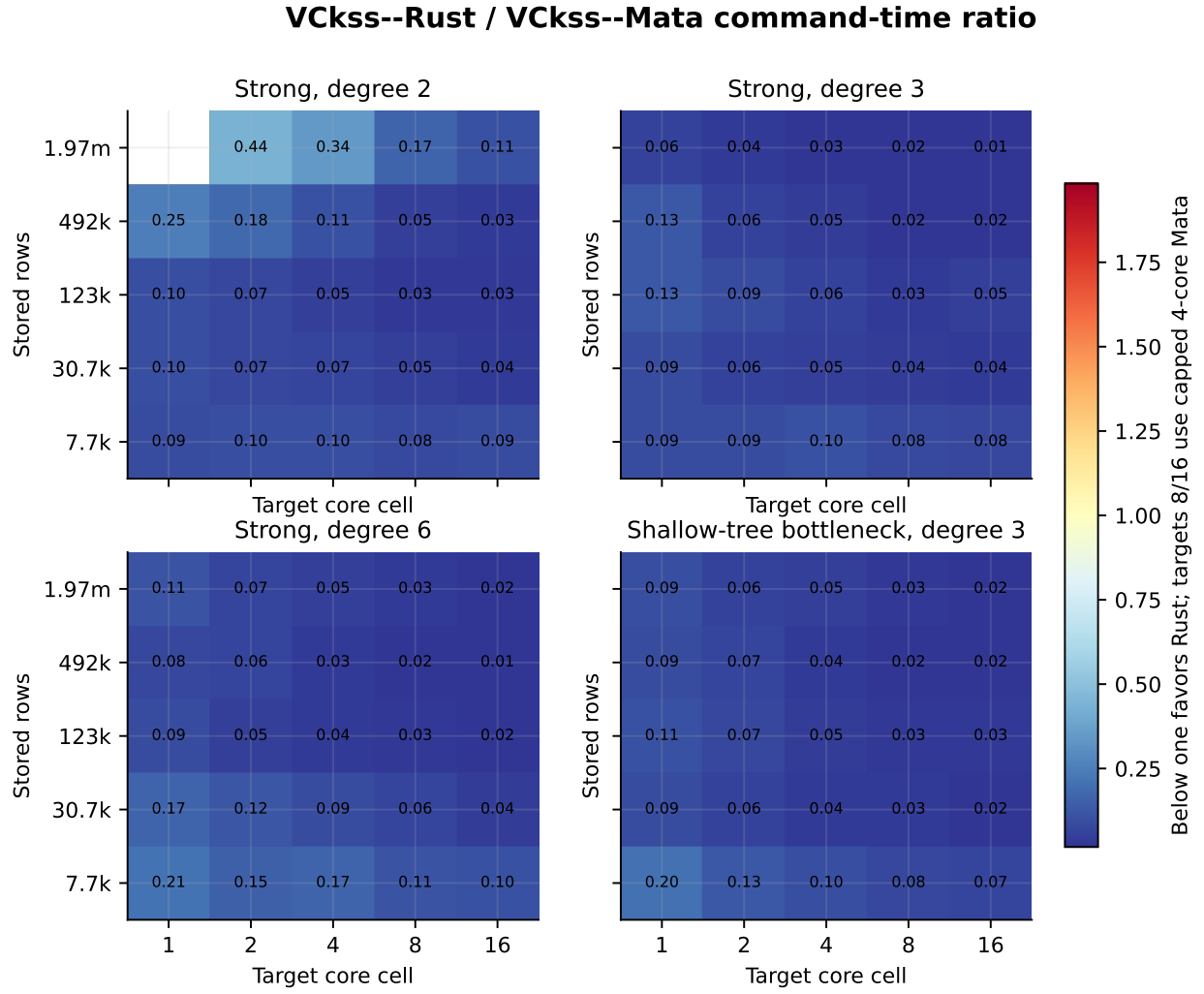


Figure 4: Median VCKss--Rust / VCKss--Mata command-time ratios. Values below one favor Rust. Target cells eight and 16 compare full-target Rust with four-core Mata and are not equal-core comparisons.

Table 2: Representative results at 1,966,080 stored rows.

Graph	Route	Target/effective cores	Time (s)	Phase GiB	Process GiB	Status
Strong d2	VCkss-Mata	1/1	—	—	—	CENSORED
Strong d2	VCkss-Rust	1/1	—	—	—	CENSORED
Strong d2	MATLAB	1/1	—	—	—	CENSORED
Strong d2	VCkss-Mata	4/4	4373.06	15.1	15.1	COMPLETE
Strong d2	VCkss-Rust	4/4	1456.19	5.54	5.54	COMPLETE
Strong d2	MATLAB	4/4	3241.49	12.2	12.2	COMPLETE
Strong d2	VCkss-Mata	16/4	4561.68	15.1	15.1	COMPLETE
Strong d2	VCkss-Rust	16/16	433.48	5.43	5.43	COMPLETE
Strong d2	MATLAB	16/16	1691.5	38.18	38.18	COMPLETE
Strong d3	VCkss-Mata	1/1	3478.12	12.95	12.95	COMPLETE
Strong d3	VCkss-Rust	1/1	210.24	3.3	3.3	COMPLETE
Strong d3	MATLAB	1/1	697.78	4.95	4.95	COMPLETE
Strong d3	VCkss-Mata	4/4	3054.35	13.73	13.73	COMPLETE
Strong d3	VCkss-Rust	4/4	76.81	4.49	4.49	COMPLETE
Strong d3	MATLAB	4/4	380.54	11.95	11.95	COMPLETE
Strong d3	VCkss-Mata	16/4	3091.66	13.7	13.7	COMPLETE
Strong d3	VCkss-Rust	16/16	31.85	4.66	4.66	COMPLETE
Strong d3	MATLAB	16/16	233.81	37.48	37.48	COMPLETE
Strong d6	VCkss-Mata	1/1	2676.2	11.49	11.49	COMPLETE
Strong d6	VCkss-Rust	1/1	301.65	3.01	3.01	COMPLETE
Strong d6	MATLAB	1/1	454.06	5.11	5.11	COMPLETE
Strong d6	VCkss-Mata	4/4	2551.23	12.22	12.22	COMPLETE
Strong d6	VCkss-Rust	4/4	136.38	3.94	3.94	COMPLETE
Strong d6	MATLAB	4/4	215.2	12.14	12.14	COMPLETE
Strong d6	VCkss-Mata	16/4	3015.43	12.29	12.29	COMPLETE
Strong d6	VCkss-Rust	16/16	56.06	4.73	4.73	COMPLETE
Strong d6	MATLAB	16/16	209.76	38.03	38.03	COMPLETE
Weak d3	VCkss-Mata	1/1	2910.02	13.5	13.5	COMPLETE
Weak d3	VCkss-Rust	1/1	276.09	4.35	4.35	COMPLETE
Weak d3	MATLAB	1/1	825.71	4.89	4.89	COMPLETE
Weak d3	VCkss-Mata	4/4	2482.19	14.39	14.39	COMPLETE
Weak d3	VCkss-Rust	4/4	119.12	5.31	5.31	COMPLETE
Weak d3	MATLAB	4/4	282.88	11.47	11.47	COMPLETE
Weak d3	VCkss-Mata	16/4	2440.61	14.31	14.31	COMPLETE
Weak d3	VCkss-Rust	16/16	42.58	5.21	5.21	COMPLETE
Weak d3	MATLAB	16/16	223.42	36.35	36.35	COMPLETE

Timing decomposition

The complete command time is the primary outcome. Process time includes the fresh application launch and all validated output handling; import and MATLAB pool management are reported separately and are never hidden inside the primary comparison. Mata’s VCkss scalar phases, Rust’s documented native phase profile, and full-CMG phases are secondary diagnostics for understanding scaling rather than alternative headline times. Legacy VCkss scalar phases are not assigned to the full-CMG early-return route.

Table 3: Role-effective scaling at the largest size; Mata is capped at four cores.

Graph	Route	Maximum effective cores	1 to max speedup	8 to 16 gain (pct)
Strong d2	VCkss–Mata	4	–	–
Strong d2	VCkss–Rust	16	–	44.02
Strong d2	MATLAB	16	–	46.44
Strong d3	VCkss–Mata	4	1.14	–
Strong d3	VCkss–Rust	16	6.6	44.01
Strong d3	MATLAB	16	2.98	2.44
Strong d6	VCkss–Mata	4	1.05	–
Strong d6	VCkss–Rust	16	5.38	-2.25
Strong d6	MATLAB	16	2.16	-97.3
Weak d3	VCkss–Mata	4	1.17	–
Weak d3	VCkss–Rust	16	6.48	31.76
Weak d3	MATLAB	16	3.7	19.75

Table 4: Representative timing components at 1,966,080 rows and four cores (seconds).

Graph	Route	Command	Process	Import	Pool start	Pool stop
Strong d2	VCkss–Mata	4373.06	4387.0	3.7	–	–
Strong d2	MATLAB	3241.49	3558.56	6.64	115.56	6.0
Strong d2	VCkss–Rust	1456.19	1470.65	5.29	–	–
Strong d3	VCkss–Mata	3054.35	3067.49	4.01	–	–
Strong d3	MATLAB	380.54	677.59	6.09	94.29	5.09
Strong d3	VCkss–Rust	76.81	89.96	3.9	–	–
Strong d6	VCkss–Mata	2551.23	2561.15	3.93	–	–
Strong d6	MATLAB	215.2	621.14	6.48	154.01	5.61
Strong d6	VCkss–Rust	136.38	149.16	4.06	–	–
Weak d3	VCkss–Mata	2482.19	2496.22	5.36	–	–
Weak d3	MATLAB	282.88	797.37	10.28	144.95	8.77
Weak d3	VCkss–Rust	119.12	134.02	5.31	–	–

Table 5: VCkss–Rust native and CMG-solve phase medians at 1,966,080 rows and four cores (seconds).

Graph	Ingest	Canonicalize	Graph build	Compress	Plan	Stayer aug.	Native solve	Native total	CMG solve
Strong d2	0.1	1.05	1.18	0.92	0.34	0.0	1450.18	1453.78	1409.09
Strong d3	0.1	1.01	0.99	0.92	0.35	0.0	71.02	74.38	46.31
Strong d6	0.09	0.94	0.81	0.91	0.33	0.0	130.76	134.02	109.53
Weak d3	0.1	0.94	1.38	0.92	0.36	0.0	112.78	116.49	48.36

3 Memory and the time–memory tradeoff

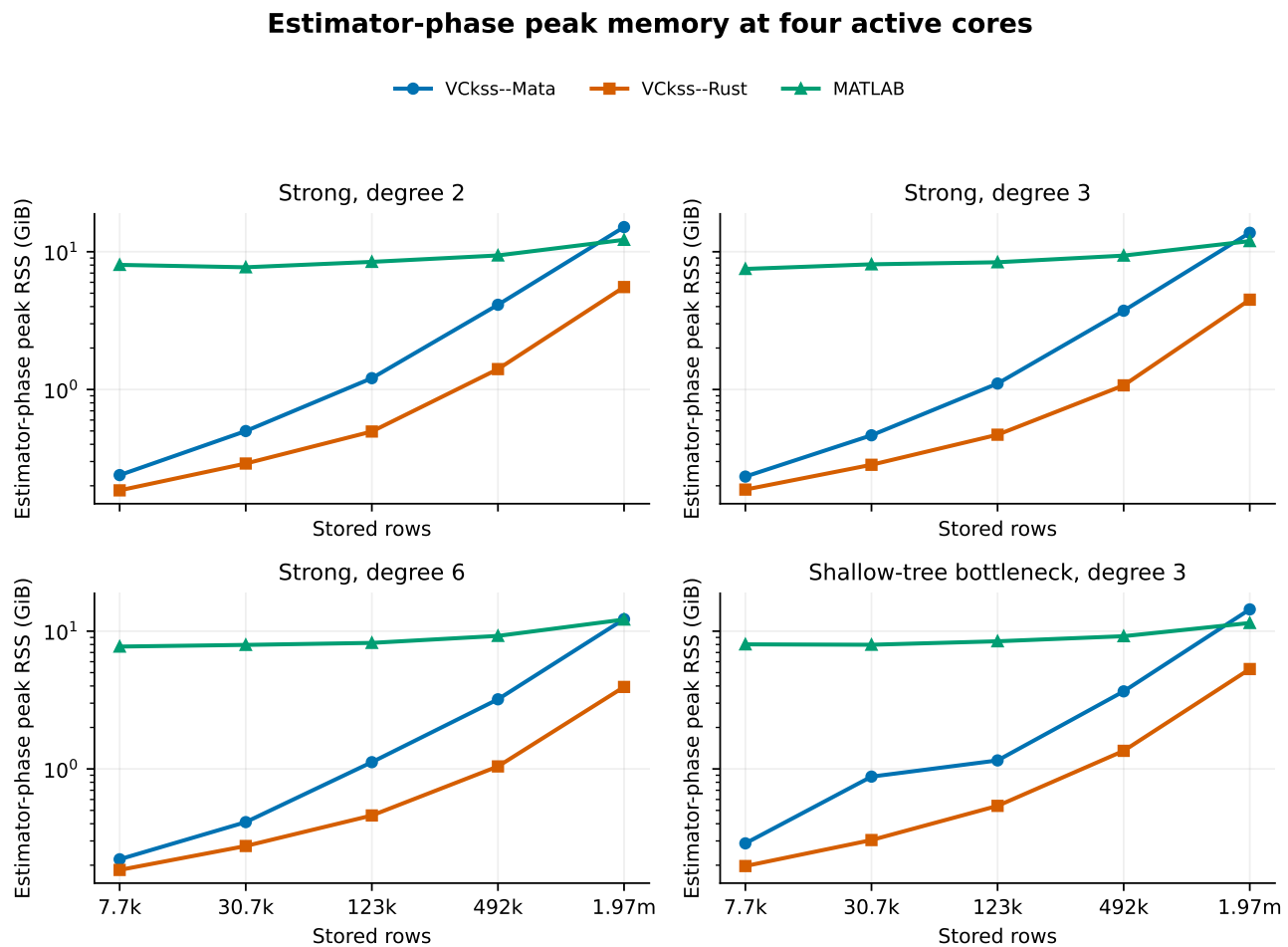


Figure 5: Estimator-phase peak summed process-tree RSS at four active cores.

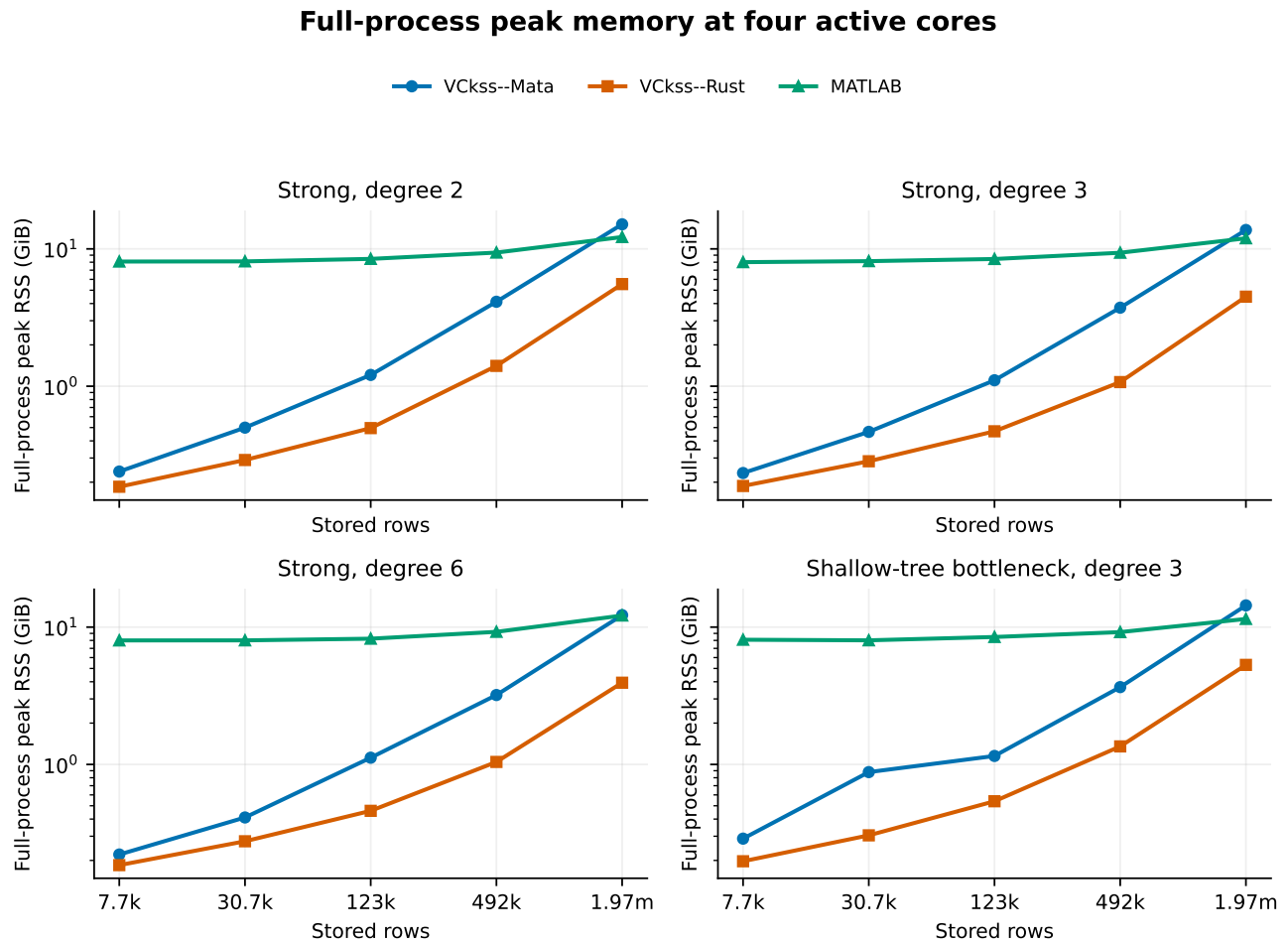


Figure 6: Full-process peak summed process-tree RSS at four active cores, including import, runtime startup, and teardown.

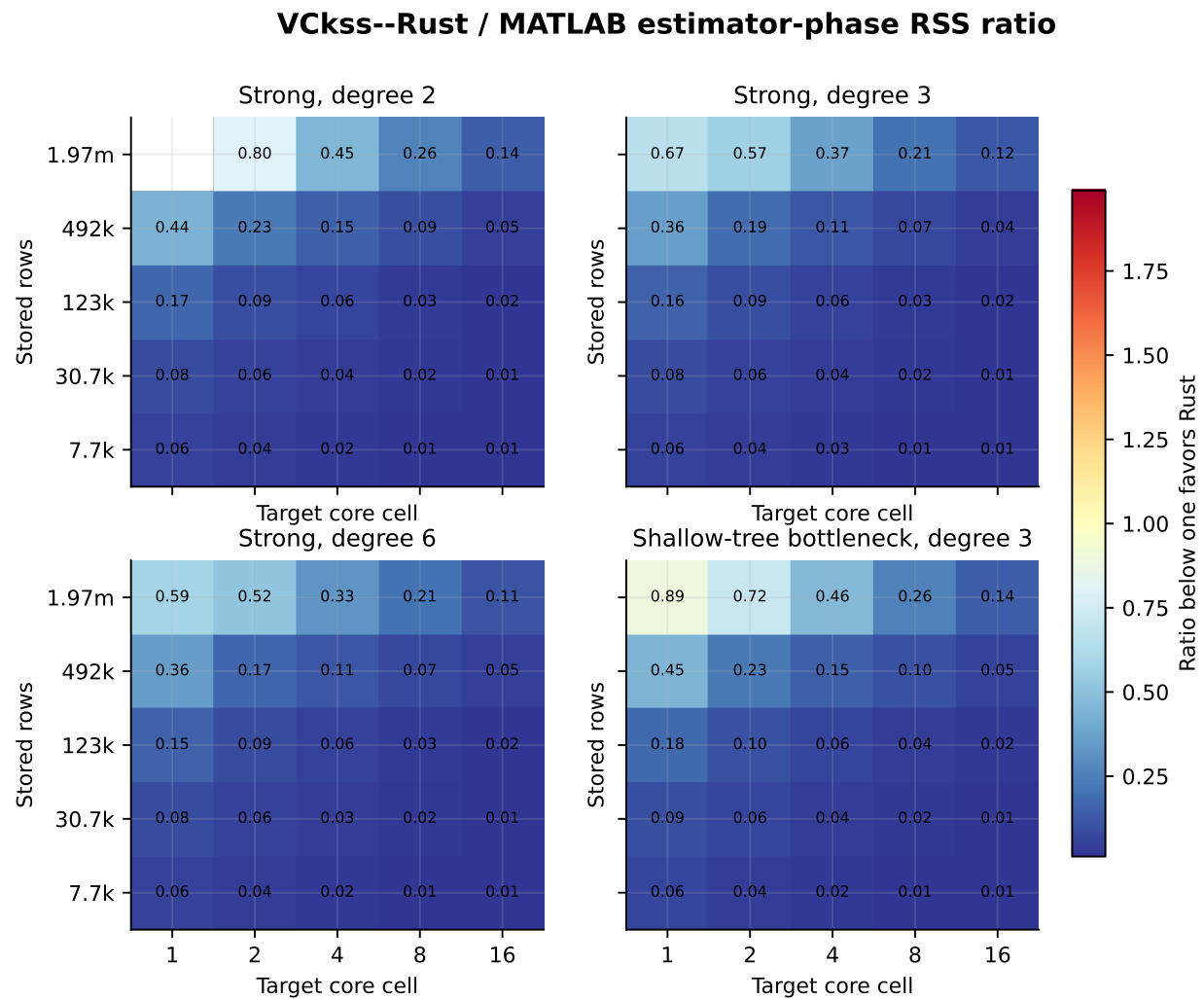
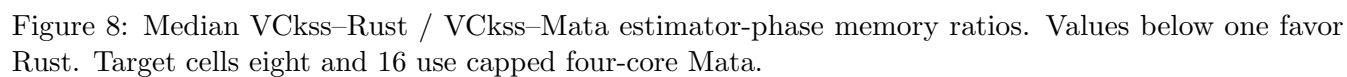


Figure 7: Median VCkss--Rust / MATLAB estimator-phase memory ratios. Values below one favor Rust.



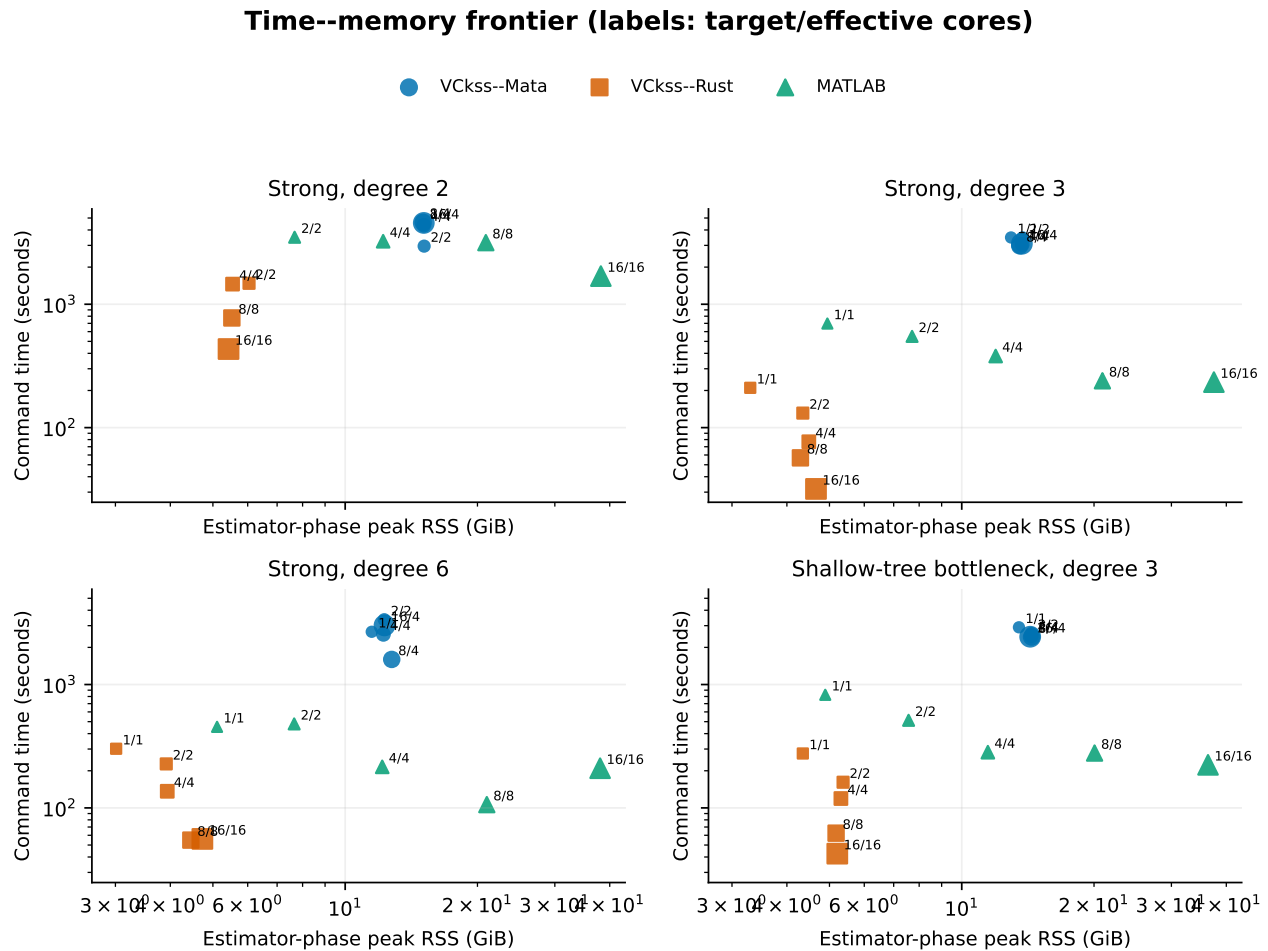


Figure 9: Time--memory points at the largest row count. Labels are target/effective core counts; marker area increases with the target.

The generated `memory_budget_guidance.tsv` reports the largest measured problem whose full-process RSS fits 8, 16, 32, or 64 GiB after a 25% safety margin. It is deliberately empirical: extrapolation outside the grid is not licensed.

4 Failures, censoring, and numerical status

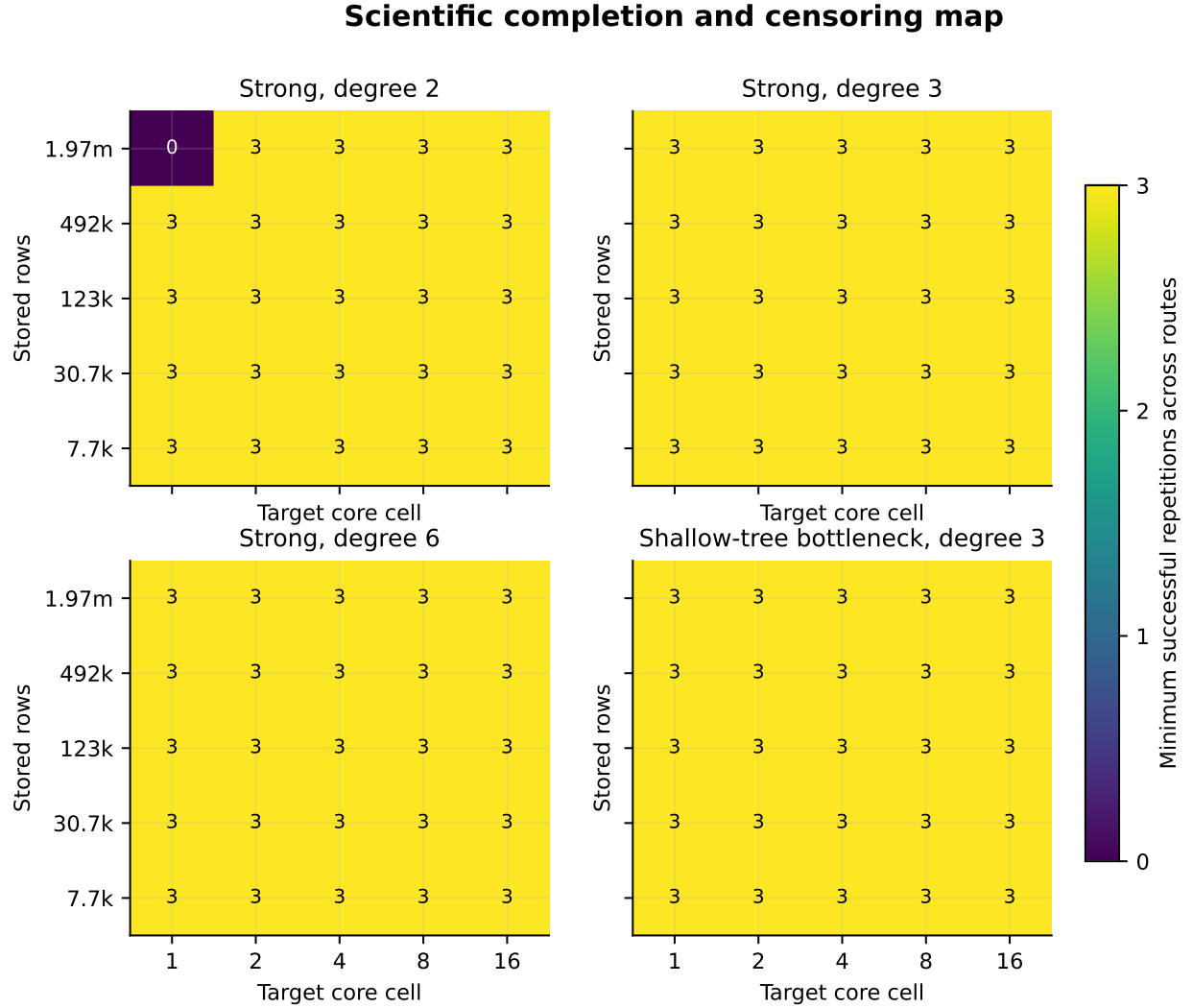


Figure 10: Minimum number of successful repetitions across the three routes. A cell requires three for ranking.

Table 6: Registered right-censored MATLAB calls.

Graph	Rows	Cores	Rep.	Route	Time evidence	Treatment
Strong d2	1966080	1	1	MATLAB	At least 10,800 s	Right-censored; cell unranked
Strong d2	1966080	1	2	MATLAB	At least 10,800 s	Right-censored; cell unranked
Strong d2	1966080	1	3	MATLAB	At least 10,800 s	Right-censored; cell unranked

Table 7: Application and scientific-status counts.

Route	scientific_status	Calls
Vckss-Mata	PASS	297
MATLAB	PASS	297
Vckss-Rust	PASS	297
MATLAB	RIGHT_CENSORED_MATLAB_TIMEOUT	3

The 891-row accepted ledger contains only complete three-route tasks. The three censored MATLAB attempts remain in a separate, source-bound ledger with their scheduler and wrapper evidence. They are displayed as lower bounds and are never used to name a winner. Three completed repetitions are summarized with their median and full range; they are not presented as a confidence interval.

5 Measurement details

Each SCC job runs one graph–size–target–core–replicate cell and all three estimators sequentially on the same scheduler-assigned host. Order rotates across repetitions. Jobs request 16 bound slots in project **welfgr** without a fixed queue, CPU model, or exclusive-node constraint. **taskset** restricts each fresh process tree to its registered role-specific subset. Rust and MATLAB use the full target; Stata and Mata use at most the first four CPUs. Stata, Rust, MATLAB pool identities, and the Linux process monitor independently verify these counts and subsets. Host, CPU model, and affinity are receipted. Paired within-task ratios are primary; absolute timing and cross-core scaling across different CPU models are descriptive unless shown for a homogeneous stratum.

Primary time is the estimator invocation measured inside the application. Secondary time covers fresh-process launch through validated output. Import, pool startup/teardown, and one-time plugin/MEX compilation are separate. Primary memory is the 250-ms sampled summed process-tree RSS between estimator phase markers; full-process RSS, GNU time, scheduler **maxvmem**, and Vckss forecast/admitted/retained memory are distinct receipts.

6 Reproducibility and identity

Vckss source commit: `ca162022833c3d6e6906c11cf4860baf0d52fd86`

Source archive SHA-256: `02d6eb6ffc96e0eb7eea1252605663e3e206c6ce177f3cf7f0e860c2b25f40f9`

Exact invocation contract

The literal per-task values for **SEED**, **MEMORY**, target and role-effective cores, CPU affinity, and execution order are in **task_manifest_300.tsv**. Each call runs in a fresh process restricted by **taskset**; Vckss

Table 8: Pinned application and runtime identities.

Component	Exact identity
Rust compiler	rustc 1.85.1 (4eb161250 2025-03-15)
Cargo	cargo 1.85.1 (d73d2caf9 2024-12-31)
Stata	stata-mp/19
MATLAB	matlab/2024b
Rust plugin SHA-256	dae0e58273f74a7d9649160ba37a99cfe1b337a8b4f05aec015fad7211bf8ec3
MATLAB source commit	8b957ffeb10b8465a3584fceb0265cccc48379e1

deliberately omits `tolerance()`. Preparation hashes a benchmark-only Ado adapter that supplies full-target Rust threads through the existing internal boundary while keeping Stata at no more than four processors. This does not change the public command or installed package.

```
vckss y, worker(worker) firm(firm) deletion(match)
    probeorder(observation_key) backend(rust) rng(counter_v1)
    algorithm(jla) engine(auto) preconditioner(auto) batch(auto)
    memory_gib(MEMORY) wallseconds(10800) probes(200)
    seed(SEED) maxiter(20000) nodisplay

vckss y, worker(worker) firm(firm) deletion(match)
    probeorder(observation_key) backend(mata) rng(stata)
    algorithm(jla) engine(auto) preconditioner(auto) batch(auto)
    memory_gib(MEMORY) wallseconds(10800) probes(200)
    seed(SEED) maxiter(20000) nodisplay

[firm_variance,covariance,worker_variance] = leave_out_KSS(
    outcome,worker,firm,[], 'matches','JLA',200,0,[],[],detail_stub);
```

The compact evidence includes the immutable 300-task manifest, the 891-call accepted ledger, the three-row censor ledger, cell summaries, the 297-task scheduler index, collection receipt, source and binary hashes, and failure evidence. Detailed logs remain in the SCC run. Exact commands are registered in `PROTOCOL.md`; the collection receipt binds every displayed figure and table to these identities.

7 Limitations

The benchmark is synthetic, uses one qualified JLA tuple, and covers the Linux SCC hosts selected by the scheduler. Windows is deferred. Graph degree changes worker and firm counts at fixed rows. MATLAB's RNG, tolerance, and finite- projection details differ. Heterogeneous CPUs limit unstratified absolute-time and cross-core claims, even though route ratios are paired within task. The eight- and 16-target Rust-Mata ratios use unequal effective core counts by design. Results guide choices only within the observed graphs, sizes, core counts, memory envelopes, versions, and scientific acceptance rules. The censored largest strong-degree-two one-core cell supplies no finite MATLAB time, no paired ratio, and no one-to-many-core scaling estimate. It is recorded as a known nonterminating cell and is excluded from future submissions unless the owner explicitly replaces this policy.